

封面

云南财经大学信息学院

网络点餐系统设计报告

学 期： 2024-2025 学年第一学期

课程名称： Web 开发技术

班 级： 计机 23-3

学 号： 202305002512

姓 名： 钟瑶姿

2025 年 1 月 1 日

目录

一、 系统简介	3
1.1 系统概述	3
1.2 系统目标	3
1.3 系统环境	3
二、 需求设计	3
2.1 功能需求	3
1. 用户功能	3
2. 管理员功能	4
3. 系统功能	5
2.2 非功能需求	5
1. 性能需求	5
2. 安全性需求	5
3. 可用性需求	6
4. 兼容性需求	6
三、 数据库设计	6
3.1 数据库选择	6
3.2 数据库表设计	6
1. 管理员表 (admin)	6
2. 菜单表 (menus)	7
3. 公告表 (notice)	8
4. 订单表 (orders)	8
5. 菜品分类表 (types)	9
6. 用户表 (users)	9
3.3 数据库关系图 (E-R 图)	10
3.4 索引设计	10
3.5 数据完整性约束	10
四、 系统设计	11
4.1 系统架构设计	12
4.1.1 总体架构	12
4.1.2 项目模块划分	13
4.1.3 技术栈	13
4.2 功能模块设计	14
4.2.1 前台用户模块 (/qiantai)	14
4.2.2 后台管理模块 (/admin)	14
4.3 配置文件设计	15
4.3.1 Spring 配置	15
4.3.2 MyBatis 配置	16
4.3.3 日志管理	16
4.4 关键组件说明	16
4.4.1 拦截器	16
4.4.2 Service 层	16
4.4.3 Mapper 文件	16
4.5 系统交互流程	17

1) 用户登录流程:	17
2) 下订单流程:	17
3) 管理员操作流程:	17
五、 界面设计、运行效果	17
5.1 前台界面设计与运行效果	17
5.1.1 首页	18
5.1.2 用户注册与登录页	18
5.1.3 菜单列表页	19
5.1.4 购物车页	20
5.2 后台界面设计与运行效果	21
5.2.1 管理员登录页	21
5.2.2 菜单管理页	21
5.2.4 订单管理页	23
5.3 设计原则与优化点	24
5.4 测试与改进建议	24

一、 系统简介

1.1 系统概述

随着互联网的发展和电子商务的兴起，传统餐饮行业也逐渐向线上转型。网络点餐系统作为餐饮行业的重要组成部分，极大地提高了餐饮服务的效率和用户体验。在外卖需求日益增长的背景下，构建一个便捷、安全、高效的网络点餐系统，已成为餐饮商家提升竞争力的重要手段。本项目旨在设计并实现一个满足基本需求的网络点餐系统，提供给用户和管理员便捷的点餐及管理体验。

1.2 系统目标

系统的主要目标如下：

- 1) 实现一个用户友好的界面，使非技术用户也能轻松地进行点餐操作。
- 2) 提供高效的菜品管理和订单处理机制，提高餐厅的运营效率。
- 3) 保障用户数据的安全，包括个人信息和支付信息。
- 4) 为管理员提供一个功能全面的后台管理系统，以便于维护和更新系统信息。

1.3 系统环境

系统运行环境要求如下：

- 1) 服务器端：JDK 8 或以上版本，Eclipse IDE for Java EE Developers，Tomcat 9.0 或以上版本作为 Web 应用服务器，MySQL 8.0 或以上版本作为数据库服务器。
- 2) 客户端：支持现代浏览器如 Chrome、Firefox、Safari 等，无需特殊插件即可访问。
- 3) 开发工具：Navicat 12 或以上版本用于数据库设计和管理。

二、 需求设计

2.1 功能需求

1. 用户功能

1) 用户注册和登录：

- 用户可以在系统中注册新账户，提供用户名、密码、电子邮件等信息。
- 用户可以登录已注册账户，通过用户名和密码进行身份验证。
- 用户可以找回密码，通过验证邮箱或手机短信进行重置。

2) 浏览菜品：

- 用户可以按菜品类别（如主食、饮料、甜点等）浏览菜品。

-每个菜品展示包括名称、图片、价格和简短描述。

3) 查看菜品详情： 用户点击菜品名称，可以查看菜品的详细信息，如食材、口味描述、营养成分等。

4) 点餐车管理：

-用户可以将选中的菜品添加到购物车，修改数量或删除购物车中的菜品。

-系统应实时更新购物车的总价格。

5) 订单提交与查看订单状态：

-用户在购物车中确认订单后，填写送货地址、选择支付方式并提交订单。

-提交订单后，用户可以查看订单的当前状态（如：待处理、已发货、已完成等）。

6) 修改用户资料： 用户可以在个人信息页修改自己的用户名、密码、电子邮件和配送地址等信息。

2. 管理员功能

1) 管理员登录： 管理员使用用户名和密码登录后台管理系统，进行数据管理和系统操作。

2) 菜品管理：

-管理员可以对菜品进行增、删、改、查操作。

-管理员可以设置菜品的名称、价格、图片、描述、所属类别等信息。

3) 用户管理： 管理员可以查看所有用户的信息，进行必要的操作（如：冻结账户、删除用户等）。

4) 订单管理：

-管理员可以查看所有用户的订单，查看每个订单的详细信息，包括订单内容、金额、状态等。

-管理员可以修改订单的状态，如：从“待处理”更新为“已发货”。

5) 菜品分类管理： 管理员可以新增、删除或修改菜品类别（如：主食、饮品、热菜等），确保菜品分类的清晰和准确。

3. 系统功能

1) 用户和管理员身份验证:

-系统通过身份验证确保只有合法用户和管理员能够访问其相关的功能模块。

-采用加密存储用户密码，确保用户隐私和账户安全。

2) 订单支付接口：系统支持简化的支付接口，用户在提交订单时可以选择支付方式（如线上支付、货到付款等）。

3) 数据统计与报告：系统应支持生成基本的统计报告，例如：每日订单量、菜品销量、用户活跃度等，为管理员提供运营决策支持。

2.2 非功能需求

1. 性能需求

1) 高并发支持:

-系统需要支持高并发的请求处理，能够在用户量较大的情况下仍然保证响应速度和系统稳定性。

-系统应该能同时处理至少 1000 个并发用户的操作请求，确保在高峰期如餐点时间不发生崩溃。

2) 响应时间要求：用户的请求响应时间应控制在 3 秒以内，确保用户体验流畅，减少用户流失。

2. 安全性需求

1) 数据加密:

-所有用户的密码、订单详情等敏感信息应使用加密技术存储，防止信息泄露。

-系统应采用 SSL/TLS 协议确保用户与服务器之间的数据传输加密。

2) 身份验证与授权:

-用户和管理员登录应通过身份验证，防止未授权访问。

-系统应实现基于角色的权限管理，确保不同角色访问不同的功能。

3. 可用性需求

1) **系统可用性：** 系统的年可用性需达到 99.9%以上，确保大部分时间系统都能正常运行。

2) 界面友好性：

-系统界面需要简洁、直观，易于用户操作。对于初次使用的用户，应该提供简单明了的操作指导。

-前端页面需适配不同屏幕尺寸，确保用户在手机、平板和电脑等设备上均能良好使用。

4. 兼容性需求

1) **浏览器兼容：** 系统应支持主流浏览器（如 Chrome、Firefox、Safari 等）的访问，并保证在不同浏览器中的表现一致。

2) **移动端支持：** 系统应优化为响应式设计，确保在手机或平板等移动设备上的浏览和操作流畅。

三、 数据库设计

在本系统中，使用 MySQL 作为数据库管理系统，数据库的设计是支持系统高效运行的基础。为了存储与点餐相关的所有信息，我们设计了多个表，涵盖了管理员、菜单、用户、订单等数据结构。

3.1 数据库选择

本系统选择了 **MySQL** 作为关系型数据库管理系统（RDBMS）。MySQL 具有高性能、可靠性、易用性和扩展性，适用于本系统的开发需求。通过 SQL 语言实现数据的增删改查操作，并利用外键和约束保证数据的完整性。

3.2 数据库表设计

根据系统需求，设计了以下主要的表结构：

1. 管理员表（admin）

用于存储管理员的基本信息，如账号、密码和权限等。

```

DROP TABLE IF EXISTS `admin`;
CREATE TABLE `admin` (
  `id` int(4) NOT NULL AUTO_INCREMENT,
  `name` varchar(20) NOT NULL,
  `pwd` varchar(20) DEFAULT NULL,
  `authority` varchar(10) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8;

```

字段说明：

- id: 管理员的唯一标识，主键，自增长。
- name: 管理员的用户名。
- pwd: 管理员的密码。
- authority: 管理员的权限等级。

2. 菜单表 (menus)

用于存储菜品的详细信息，包括菜品名称、类型、价格、库存等。

```

DROP TABLE IF EXISTS `menus`;
CREATE TABLE `menus` (
  `id` int(4) unsigned NOT NULL AUTO_INCREMENT,
  `name` varchar(20) NOT NULL,
  `typeid` int(4) unsigned DEFAULT NULL,
  `burden` varchar(50) DEFAULT NULL,
  `brief` varchar(500) DEFAULT NULL,
  `price` float unsigned DEFAULT NULL,
  `sums` int(4) unsigned DEFAULT '0',
  `price1` float unsigned DEFAULT NULL,
  `sums1` int(4) unsigned DEFAULT '0',
  `imgpath` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=20 DEFAULT CHARSET=utf8;

```

字段说明：

- id: 菜品的唯一标识，主键，自增长。
- name: 菜品名称。
- typeid: 菜品类型 ID，外键关联 types 表。
- burden: 菜品的原料。
- brief: 菜品描述。
- price: 菜品标准价格。
- sums: 菜品的当前库存数量。
- price1: 优惠后的价格（如果有）。
- sums1: 优惠菜品的库存数量。
- imgpath: 菜品图片的路径。

3. 公告表 (notice)

用于存储系统的公告信息，管理员可在此表中发布重要通知或新菜单的上线信息。

```
DROP TABLE IF EXISTS `notice`;
CREATE TABLE `notice` (
  `id` int(4) NOT NULL AUTO_INCREMENT,
  `name` varchar(100) NOT NULL,
  `content` varchar(2000) DEFAULT NULL,
  `times` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8;
```

- 字段说明：
 - id: 公告的唯一标识，主键，自增长。
 - name: 公告标题。
 - content: 公告的详细内容。
 - times: 公告发布的时间。

4. 订单表 (orders)

用于存储订单的详细信息，包括用户信息、菜单项、数量、时间等。

```
DROP TABLE IF EXISTS `orders`;
CREATE TABLE `orders` (
  `id` int(4) NOT NULL AUTO_INCREMENT,
  `userid` int(4) unsigned DEFAULT NULL,
  `menuid` int(4) unsigned DEFAULT NULL,
  `menusum` int(4) unsigned DEFAULT NULL,
  `times` varchar(20) DEFAULT NULL,
  `delivery` int(4) unsigned DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=29 DEFAULT CHARSET=utf8;
```

- 字段说明：
 - id: 订单的唯一标识，主键，自增长。
 - userid: 外键，关联 users 表，表示该订单属于哪个用户。
 - menuid: 外键，关联 menus 表，表示该订单中包含的菜品。
 - menusum: 该菜品的数量。
 - times: 订单创建时间。
 - delivery: 配送方式（如：自取、送货上门等）。

5. 菜品分类表 (types)

用于存储菜品的分类信息，便于管理和浏览。

```
DROP TABLE IF EXISTS `types`;
CREATE TABLE `types` (
  `id` int(4) NOT NULL AUTO_INCREMENT,
  `name` varchar(20) NOT NULL,
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=12 DEFAULT CHARSET=utf8;
```

- 字段说明：
 - id: 菜品分类的唯一标识，主键，自增长。
 - name: 菜品分类名称（如：凉拌菜、炒菜等）。

6. 用户表 (users)

用于存储用户的基本信息，包括用户名、密码、联系方式等。

```
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` (
  `id` int(4) NOT NULL AUTO_INCREMENT,
  `name` varchar(20) NOT NULL,
  `pwd` varchar(20) DEFAULT NULL,
  `realname` varchar(20) DEFAULT NULL,
  `sex` varchar(10) DEFAULT NULL,
  `age` int(4) unsigned DEFAULT NULL,
  `card` varchar(20) DEFAULT NULL,
  `address` varchar(100) DEFAULT NULL,
  `phone` varchar(20) DEFAULT NULL,
  `email` varchar(20) DEFAULT NULL,
  `code` varchar(10) DEFAULT NULL,
  `type` int(4) unsigned DEFAULT '0',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE KEY `id` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8;
```

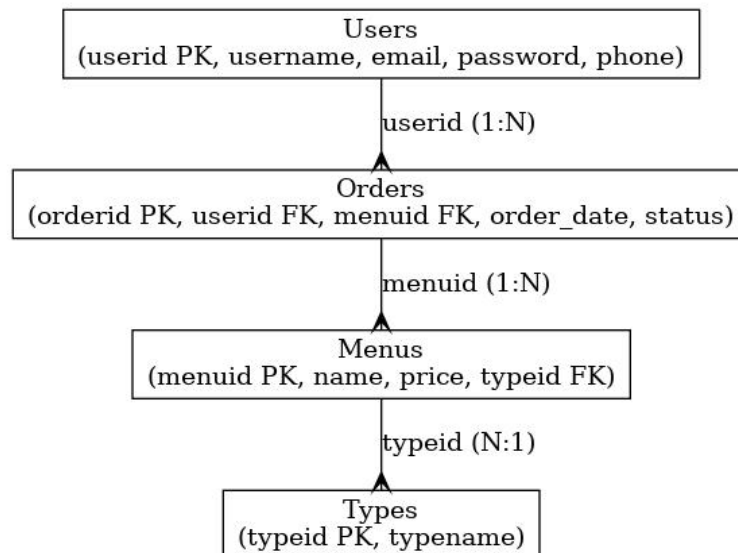
- 字段说明：
 - id: 用户的唯一标识，主键，自增长。
 - name: 用户的登录用户名。
 - pwd: 用户的密码。
 - realname: 用户的真实姓名。
 - sex: 用户性别。
 - age: 用户年龄。
 - card: 用户身份证号。

- address: 用户的联系地址。
- phone: 用户的联系电话。
- email: 用户的电子邮件地址。
- code: 用户的验证码。
- type: 用户的类型（如：普通用户、管理员等）。

3.3 数据库关系图（E-R 图）

根据以上的表结构，数据库表之间的关系如下：

- users 表 与 orders 表: 通过 userid 关联, 表示每个订单都属于某个用户。
- orders 表 与 menus 表: 通过 menuid 关联, 表示一个订单包含多个菜品。
- menus 表 与 types 表: 通过 typeid 关联, 表示每个菜品属于某一类型。



3.4 索引设计

为了提高数据库查询效率，我们为以下字段添加了索引：

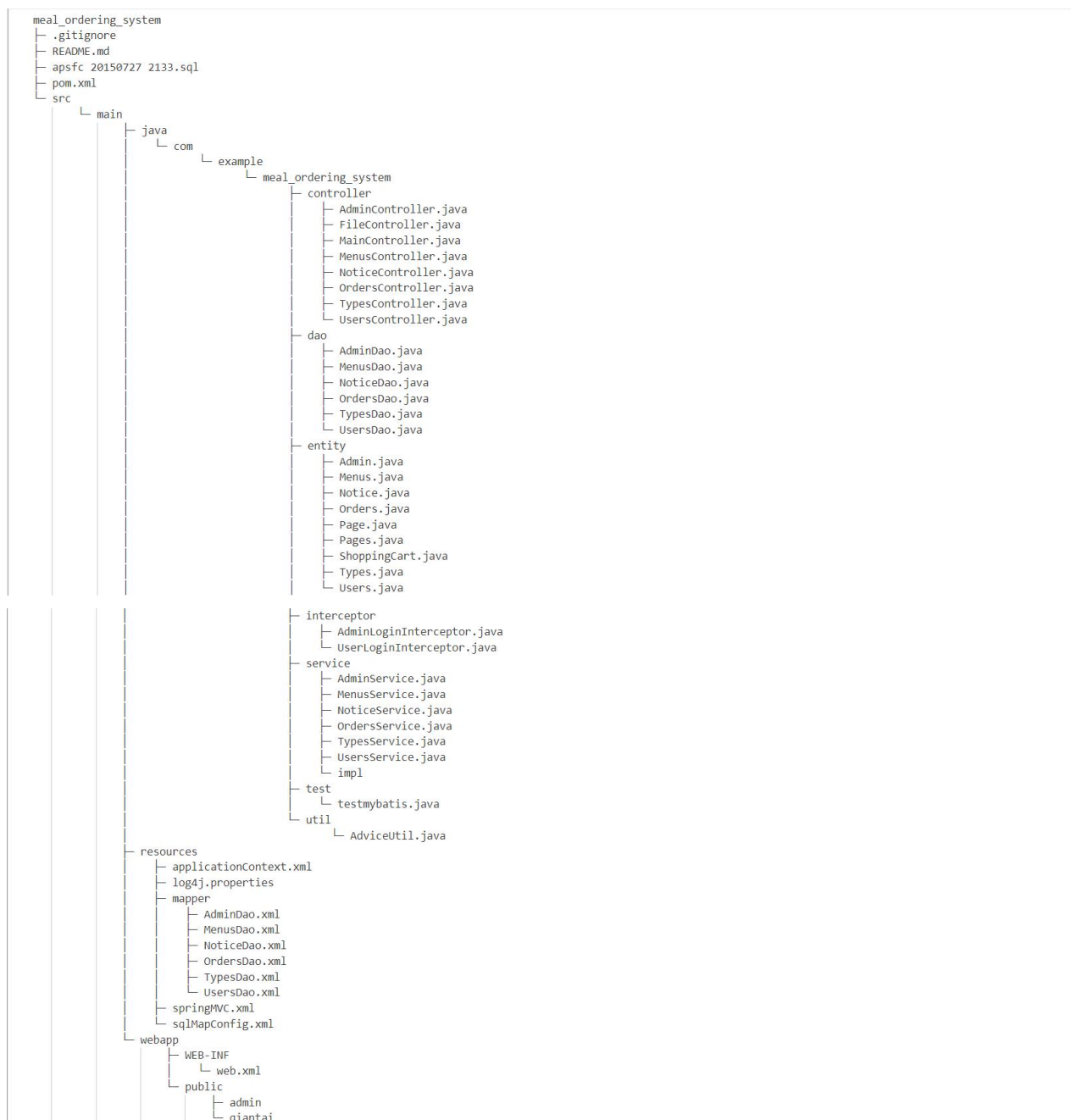
- users 表: name, phone, email 字段的索引，方便用户登录和查询。
- menus 表: name, price 字段的索引，方便快速查找菜品。
- orders 表: userid, menuid 字段的索引，优化查询订单内容。

3.5 数据完整性约束

为了确保数据的一致性和完整性，系统采用了如下的约束：

- **外键约束：**确保 orders 表中的 userid 和 menuid 字段与 users 和 menus 表中的记录一致。
- **唯一性约束：**users 表中的 name、email 字段确保唯一性，避免重复注册。
- **非空约束：**确保字段如 pwd、name、address 等不能为空，确保数据的完整性。

四、 系统设计



1. Src 下的 java 文件存放的我们后端的代码

2. Controller 主要是调用了 service 层的方法，和一些业务逻辑的编写，我们可以根据具体方法名称看出来这个方法具体的功能，比如 AdminController，里面的 update 方法就是对管理员信息的修改，toLogin 跳转到管理员登录页面，login 对前端传来参数的判断，成功或失败都跳转到对应的页面，logout 退出登录，下面的 controller 根据方法名就可以判断具体功能。
3. Dao 下面都是一些接口，里面的方法都是针对数据库操作的，他跟下面 resource 下的 mapper 里的 mapper.xml 是对应的，mapper.xml 里都是 sql 语句，dao 里面的每个方法都在 mapper.xml 有具体对数据库操作的实现。
4. Entity 里都是我们封装的实体类，基本上一个数据库的表对应着一个实体类，其他的比如 page 和 pages 是分页功能需要的实体类，ShoppingCart 是用户在下单后存放餐车需要的实体类。
5. Interceptor 里这两个类起到一个拦截的功能，分别对前台未登录的用户和后台未登录的用户进行拦截并拒绝访问的操作。
6. 再往下的 service，它里面有一些接口和这些接口的实现类，这些接口基本上都是一些增删改查的方法，然后实现类来做具体的实现，这些实现类都导入了 dao 层的接口，调用 dao 层的方法来对数据库进行具体的操作。
7. utils 就是我们的工具类了。
8. Resource 文件存放这我们的配置文件和 mapper.xml 的代码。
9. ApplicationContext.xml 文件主要配置我们 spring 注解的包扫描注释，数据库连接的配置，dao 层接口的配置，对事物和 aop 的配置。
10. Log4j 就是日志的配置
11. springMVC.xml 主要配置了我们静态资源访问路径，controller 层的扫描还有加载视图解析器和文件解析器
12. sqlMapConfig.xml 主要配置了实体类的包扫描和 dao 层的包扫描
13. webapp 主要存放着我们前端的资源，下面的 public 里的 admin 是我们后台页面需要的资源，images 包含着前台页面菜品的图片，qiantai 文件下都是我们前台页面需要的资源了
14. 最后 web-inf 里的 web.xml 主要配置了 listner, servlet 和解决中文乱码的过滤器

本系统采用基于 **SSM 框架** 的设计，结合 **MVC 架构** 和 **三层架构**，实现了系统的模块化开发和职责分离。系统通过前后台分离设计，同时支持用户端的点餐操作和管理员端的后台管理操作，极大地提升了系统的可维护性和扩展性。

4.1 系统架构设计

4.1.1 总体架构

本系统采用 **MVC (Model-View-Controller) 架构**，分为以下三个主要部分：

1) Controller 层：负责处理用户的 HTTP 请求，将请求数据传递给 Service 层，并返回处理后的结果视图或数据给前端。

2) Service 层：

-封装核心业务逻辑，为 Controller 层提供功能支持。

-负责调用 DAO 层实现与数据库的交互操作。

3) DAO 层：通过 MyBatis 框架与数据库交互，执行数据的增删改查操作，并将结果返回给 Service 层。

此外，本系统采用 **Web 前后端分离设计**：

-**前台路径：**/qiantai，提供用户登录、注册、查看菜单和下订单等功能接口。

-**后台路径：**/admin，提供管理员登录、菜单管理、公告管理和订单管理等功能接口。

4.1.2 项目模块划分

根据功能需求，系统划分为以下主要模块：

1) 前台模块：

-实现用户注册、登录、查看菜单、下订单等功能。

-用户通过路径 /qiantai 访问前台功能。

2) 后台模块：

-提供管理员登录、菜单管理、公告管理、订单管理等功能。

-管理员通过路径 /admin 访问后台功能。

4.1.3 技术栈

本系统技术栈包括：

1) **后端框架：**Spring MVC + Spring + MyBatis（SSM 框架）。

2) **数据库：**MySQL，用于存储用户、菜单、公告和订单等数据。

3) **日志管理：**Log4j，用于记录系统运行日志。

4) 配置文件：

-applicationContext.xml：配置 Spring 容器和依赖注入。

-springMVC.xml：配置 Spring MVC 的控制器映射和视图解析。

-sqlMapConfig.xml: 配置 MyBatis 的数据源和 SQL 映射文件。

4.2 功能模块设计

4.2.1 前台用户模块 (/qiantai)

用户通过路径 /qiantai 访问前台功能，具体功能如下：

1) 用户注册与登录：

-用户注册信息存储在 users 表中，登录时通过拦截器验证身份。

-相关接口： POST /qiantai/api/users/register: 用户注册。

POST /qiantai/api/users/login: 用户登录。

2) 查看菜单：

-用户可以浏览所有菜单，支持按分类筛选菜品。

-菜单数据存储在 menus 表中，通过 MenuService 提供服务。

-相关接口： GET /qiantai/api/menus: 获取所有菜单。

GET /qiantai/api/menus/{menuId}: 根据 ID 获取菜单详情。

3) 下订单：

-用户在购物车中选择菜品并提交订单，订单信息存储在 orders 表中。

-相关接口： POST /qiantai/api/orders: 创建订单。

GET /qiantai/api/orders/{orderId}: 查询订单详情。

4.2.2 后台管理模块 (/admin)

管理员通过路径 /admin 访问后台功能，具体功能如下：

1) 管理员登录：

-管理员登录信息验证后可访问管理功能，由拦截器保障安全。

-相关接口： POST /admin/api/admin/login： 管理员登录。

2) 菜单管理：

-管理员可以对菜单进行增删改查，菜品分类通过 `types` 表管理。

-相关接口： POST /admin/api/menus： 新增菜单。

PUT /admin/api/menus/{menuId}： 更新菜单。

DELETE /admin/api/menus/{menuId}： 删除菜单。

GET /admin/api/types： 获取菜单分类。

3) 订单管理：

-管理员可查看所有订单，并更新订单状态。

-相关接口： GET /admin/api/orders： 获取所有订单。

PUT /admin/api/orders/{orderId}： 更新订单状态。

4) 公告管理：

-管理员通过后台发布和管理公告。

-相关接口： POST /admin/api/notices： 新增公告。

GET /admin/api/notices： 查询所有公告。

DELETE /admin/api/notices/{noticeId}： 删除公告。

4.3 配置文件设计

4.3.1 Spring 配置

1) applicationContext.xml：

-配置 Spring 容器，加载 Service 层和 DAO 层的 Bean。

-配置数据源和 MyBatis 的会话工厂。

2) springMVC.xml：

-配置 Spring MVC 的视图解析器，将逻辑视图名称解析为 JSP 文件。

-配置静态资源处理，确保 /public 文件夹下的静态资源（如 CSS 和 JS）可被访问。

4.3.2 MyBatis 配置

sqlMapConfig.xml:

-配置 MyBatis 的全局属性和数据源。

-引用 Mapper 文件（如 MenusDao.xml）实现 SQL 映射。

4.3.3 日志管理

1) log4j.properties:

-配置日志输出级别（如 INFO、ERROR）。

-配置日志文件的存储位置和格式。

4.4 关键组件说明

4.4.1 拦截器

1) AdminLoginInterceptor.java: 负责拦截后台未登录用户的请求，确保管理员操作的安全性。

2) UserLoginInterceptor.java: 负责拦截前台未登录用户的请求，确保只有登录用户才能进行下单等操作。

4.4.2 Service 层

系统中的业务逻辑由 Service 层负责处理：

- AdminService.java: 处理管理员登录验证逻辑。
- MenusService.java: 处理菜单的增删改查逻辑。
- OrdersService.java: 处理订单的创建、查询和更新逻辑。
- TypesService.java: 处理菜单分类管理逻辑。
- UsersService.java: 处理用户注册、登录和信息管理逻辑。

4.4.3 Mapper 文件

Mapper 文件定义了 MyBatis 的 SQL 映射：

- AdminDao.xml: 管理管理员表（admin）的 SQL 操作。
- MenusDao.xml: 管理菜单表（menus）的 SQL 操作。
- OrdersDao.xml: 管理订单表（orders）的 SQL 操作。

4.5 系统交互流程

1) 用户登录流程:

- 用户提交登录表单至 /qiantai/api/users/login。
- UserLoginInterceptor 拦截请求并验证身份。
- 验证通过后，返回登录成功信息，并存储用户会话。

2) 下订单流程:

- 用户选择菜品后提交订单至 /qiantai/api/orders。
- Controller 调用 OrdersService 创建订单，并写入 orders 表。
- 返回订单创建成功的信息。

3) 管理员操作流程:

- 管理员登录 /admin/api/admin/login。
- AdminLoginInterceptor 验证管理员身份。
- 验证通过后，管理员可执行菜单、公告和订单管理操作。

本系统通过 SSM 框架实现前后台分离设计，结合拦截器、服务层逻辑和持久化操作完成功能开发。前台功能满足用户点餐需求，后台功能支持管理员高效管理，为系统的稳定性和可扩展性提供了保障。

五、 界面设计、运行效果

界面设计与运行效果展示系统功能的用户体验和实现成果，分为前台界面和后台界面两个部分，结合实际功能需求，详细说明页面布局、设计风格及关键运行效果。

5.1 前台界面设计与运行效果

前台界面主要供普通用户使用，提供注册登录、浏览菜品、下单和订单管理等功能。以下是前台主要页面的设计与运行效果说明：

5.1.1 首页

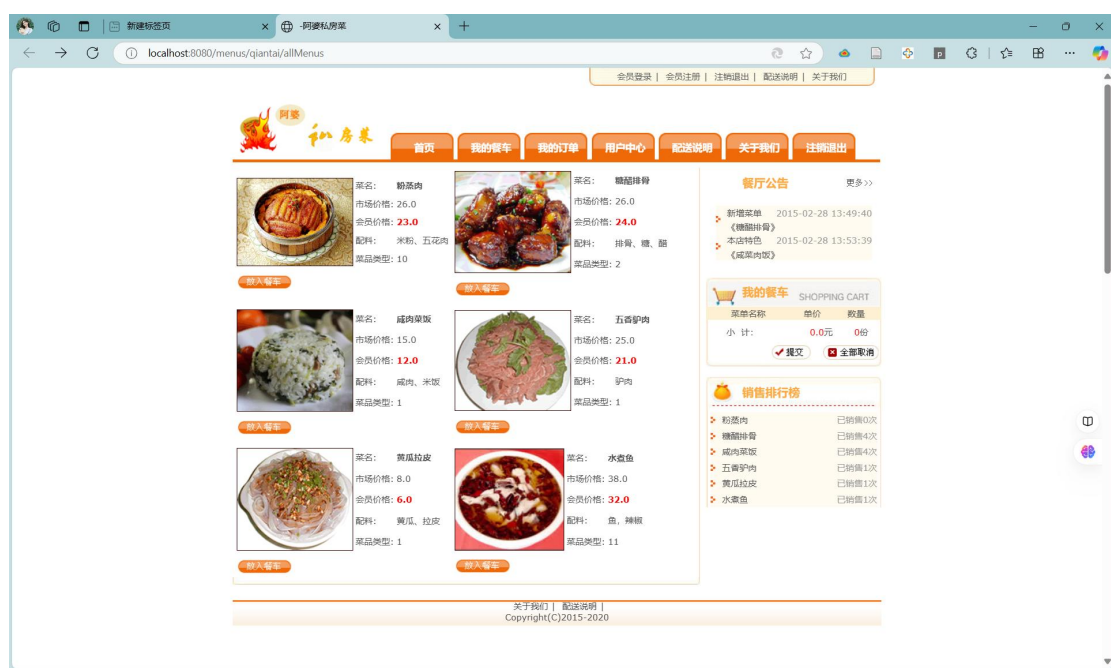
1) 设计功能:

-菜品分类展示, 用户可根据分类筛选菜品。

-推荐菜品或热门菜品区域, 吸引用户关注。

2) 运行效果:

用户访问 <http://localhost:8080/qiantai/allMenus> 时, 首页展示菜单的分类及菜品列表。



5.1.2 用户注册与登录页

1) 设计功能:

-注册页面包含用户名、密码、真实姓名、电话号码等字段, 校验必填项和格式。

-登录页面提供用户名与密码输入框, 支持错误提示和跳转。

2) 运行效果:

-注册成功后, 页面提示“注册成功”, 并跳转至登录页面。

-用户输入有效的用户名和密码后, 页面跳转至首页, 同时显示登录用户的欢迎信息。



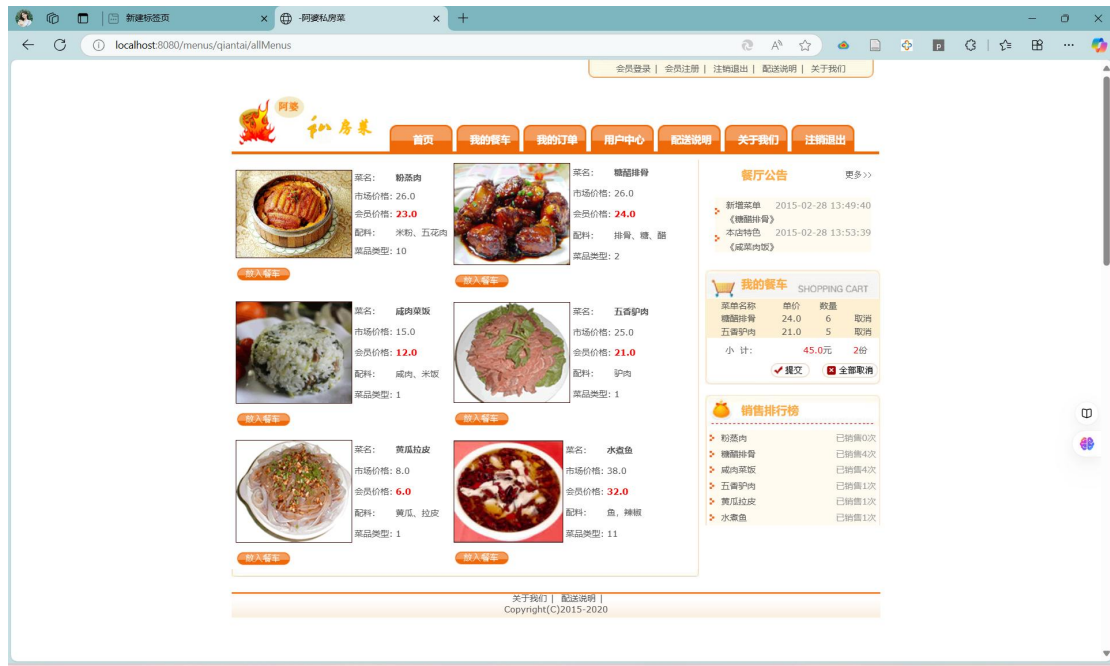
5.1.3 菜单列表页

1) 设计功能:

- 菜单以卡片式布局展示，每张卡片显示菜品图片、名称、价格和简介。
- 提供“加入购物车”按钮，用户可选择菜品并添加至购物车。

2) 运行效果:

- 用户浏览菜单时，页面动态加载数据，显示菜品分类。
- 点击“加入购物车”后，系统更新购物车的内容，并实时显示购物车中商品的数量和总价。



5.1.4 购物车页

1) 设计功能:

- 购物车页面展示用户已选择的菜品清单，包括菜品名称、数量、单价和总价。
- 提供修改数量、删除商品的功能。

2) 运行效果:



- 用户在购物车中可以增加或减少商品数量，系统实时计算总价。

-点击“提交订单”后，跳转到登录页面，要求用户先进行登录操作。

5.2 后台界面设计与运行效果

后台界面主要供管理员使用，提供登录、菜单管理、订单管理、公告管理等功能。以下是后台主要页面的设计与运行效果说明：

5.2.1 管理员登录页

1) 设计功能：输入管理员用户名和密码，验证身份后进入后台管理系统。

2) 运行效果：

-用户访问 <http://localhost:8080/admin/login>，进入管理员登录页面。

-登录成功后，系统跳转至后台首页，显示当前待处理订单数量和菜品统计信息。



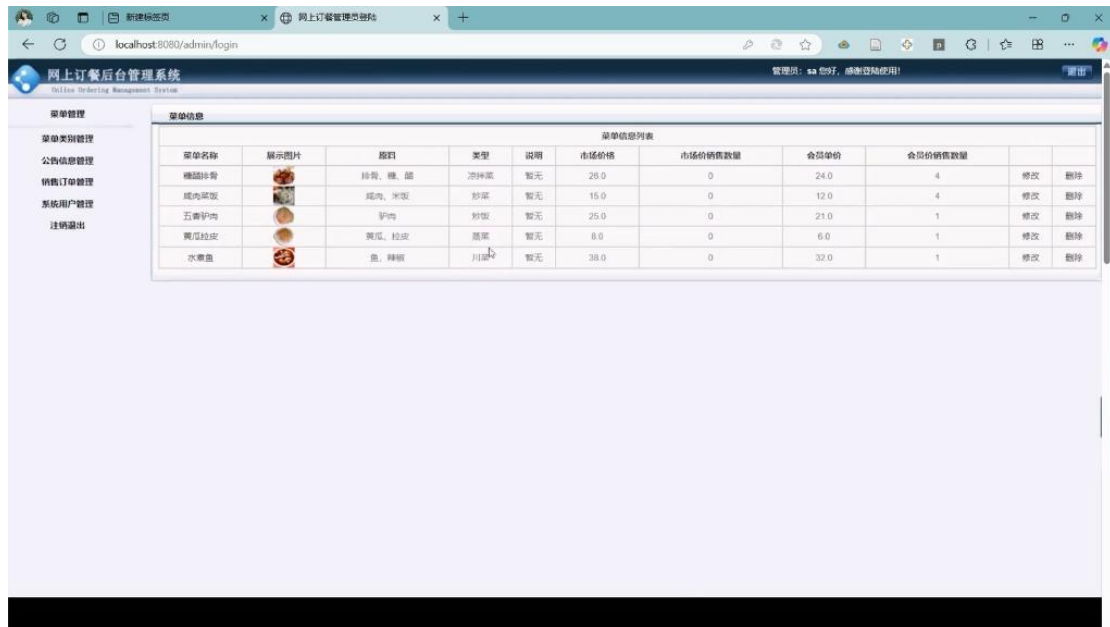
5.2.2 菜单管理页

1) 设计功能：

-管理员可在此页面新增、更新、删除菜单，并查看所有菜品分类。

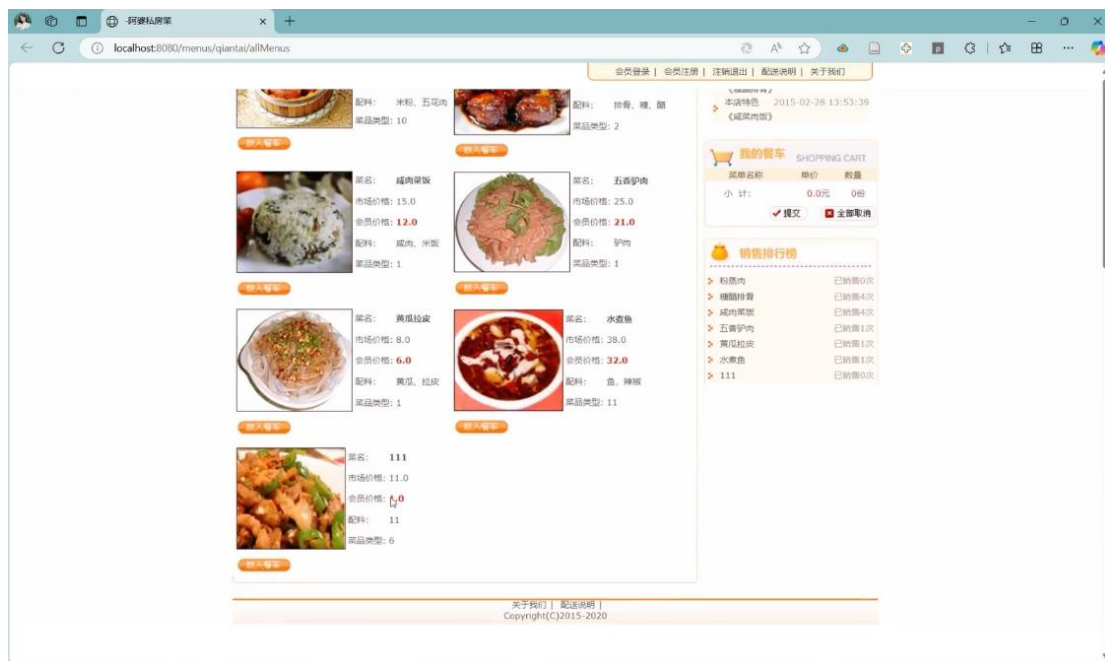
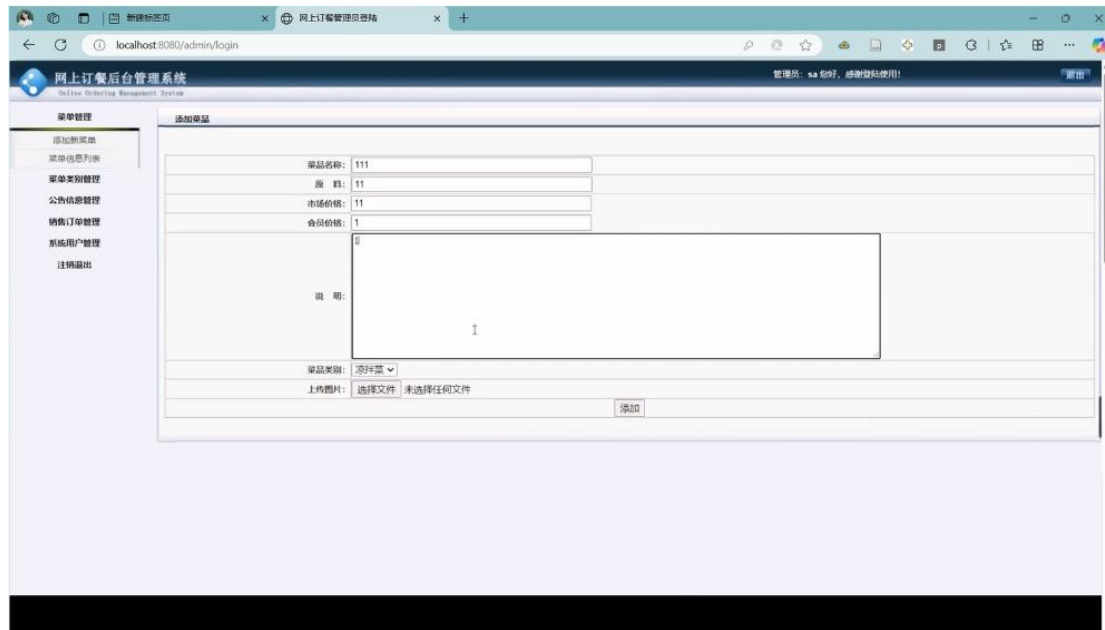
2) 运行效果：

- 用户访问 <http://localhost:8080/admin/login>，管理员登录后进入菜单管理页面。
- 管理员填写公告标题和内容后，点击“发布”，公告即刻显示在前台首页公告区域。
- 在删除菜品分类时，系统会弹出确认对话框，以避免误删除。



3) 添加菜单为例

- 管理员可以通过该页面添加新的菜品到菜单中，指定菜品名称、原料、市场价格、会员价格等信息。
- 页面包含一个表单，成功添加菜品后系统会跳转回菜单管理页面，并在菜品列表中展示新添加的菜品。



5.2.4 订单管理页

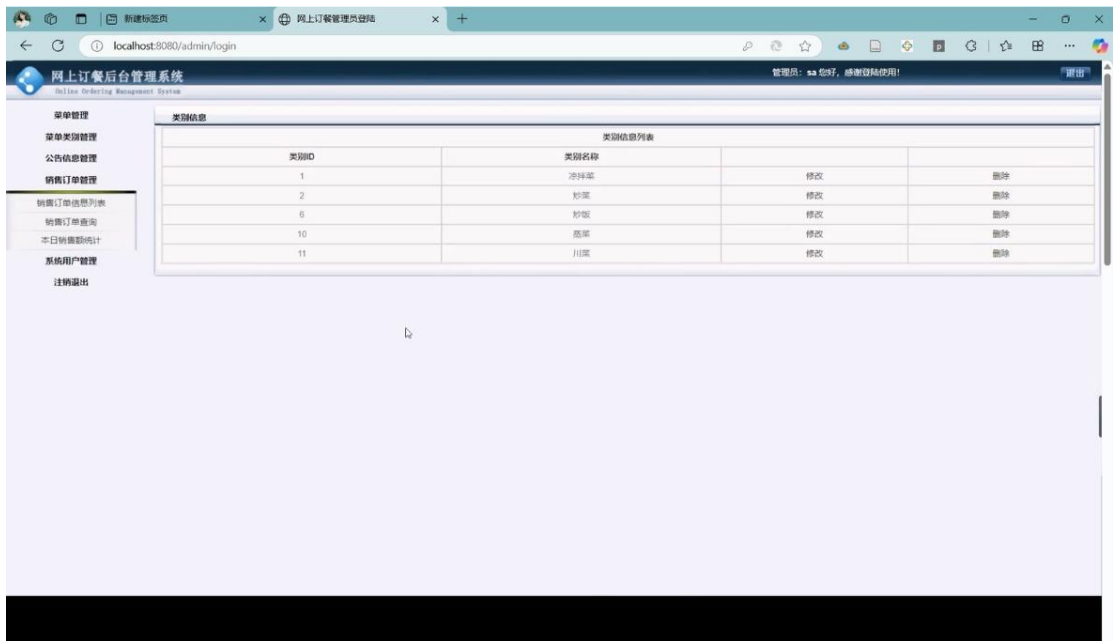
1) 设计功能:

-管理员可以查看所有订单，并能够更新订单状态。

2) 运行效果:

-用户访问 <http://localhost:8080/admin/login>，进入订单管理页面。

-页面展示所有订单的列表，包括类别 ID、类别名称、是否修改等。



5.3 设计原则与优化点

- **响应式设计：**所有页面支持多端适配，在桌面端和移动端均能良好显示。
- **用户友好性：**提供详细的提示信息（如登录错误提示、数据验证反馈等）。
- **动态交互效果：**购物车、订单状态等支持实时更新，提升用户体验。
- **后台安全性：**管理员登录页面采用拦截器验证，确保未授权用户无法访问后台功能。

5.4 测试与改进建议

- **功能测试：**针对每个页面功能点（如表单验证、动态加载等）进行单元测试，确保功能完整。
- **用户体验测试：**模拟真实用户的操作场景，观察页面是否存在卡顿、数据加载延迟等问题。
- **性能优化：**如菜单列表加载大量数据时，可采用分页加载或异步加载的方式，避免页面阻塞。